

i Front matter

Department of Electronic Systems

Examination paper for TFE4141 Design of Digital Systems 1

Examination date: December 8, 2021

Examination time (from-to): 09:00 – 13:00

Permitted examination support material: A / All printed and hand-written support material is allowed. All calculators are allowed.

Academic contact during examination: Per Gunnar Kjeldsberg
Phone: 934 59 550

OTHER INFORMATION

Get an overview of the question set before you start answering the questions.

Read the questions carefully and make your own assumptions. If a question is unclear/vague, make your own assumptions and specify them in your answer. Only contact academic contact in case of errors or insufficiencies in the question set. Address an invigilator if you wish to contact the academic contact. Write down the question in advance.

InspiraScan: For question **14** it is possible to submit the entire/some parts of the answer as handwritten sheets. At the bottom of the question you will find a seven-digit code. Fill in this code in the top left corner of the sheets you wish to submit. We recommend that you do this during the exam. If you require access to the codes after the examination time ends, click “Show submission”.

Weighting: The total number of achievable points is 100. The maximum number for each question is given on each question cover page.

Notifications: If there is a need to send a message to the candidates during the exam (e.g. if there is an error in the question set), this will be done by sending a notification in Inspira. A dialogue box will appear. You can re-read the notification by clicking the bell icon in the top right-hand corner of the screen.

Withdrawing from the exam: If you become ill or wish to submit a blank test/withdraw from the exam for another reason, go to the menu in the top right-hand corner and click “Submit blank”. This cannot be undone, even if the test is still open.

Access to your answers: After the exam, you can find your answers in the archive in Inspira. Be aware that it may take a working day until any hand-written material is available in the archive.

1 Simulation result 1 (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the following VHDL code, which alternative reflects a correct simulation result?

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity EvalTest is
    Port ( insig1, insig2, insig3 : in STD_LOGIC;
          outsig : out STD_LOGIC );
end EvalTest;

architecture Behavioral of EvalTest is
    signal eval : STD_LOGIC_VECTOR(2 downto 0);
begin
    eval <= insig1 & insig2 & insig3;
    outsig <= '0' when eval = "001" else
               '0' when eval = "010" else
               '0' when eval = "100" else
               '1';
end Behavioral;

```

Select one alternative:

☐

Name	Value	0.000 ns	10.000 ns	20.000 ns	30.000 ns	40.000 ns	50.000 ns
insig1	1						
insig2	0						
insig3	1						
outsig	0						

☐

Name	Value	0.000 ns	10.000 ns	20.000 ns	30.000 ns	40.000 ns	50.000 ns
insig1	1						
insig2	0						
insig3	1						
outsig	0						

☒

Name	Value	0.000 ns	10.000 ns	20.000 ns	30.000 ns	40.000 ns	50.000 ns
insig1	1						
insig2	0						
insig3	1						
outsig	1						

Maximum marks: 2

2 Simulation result 2 (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the following VHDL code, which alternative reflects a correct simulation result?

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

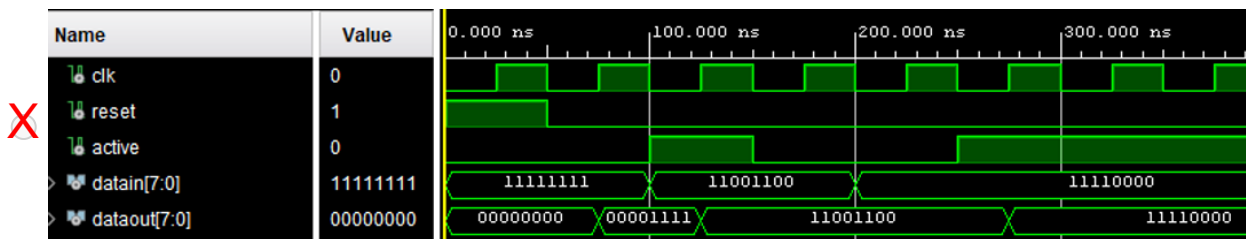
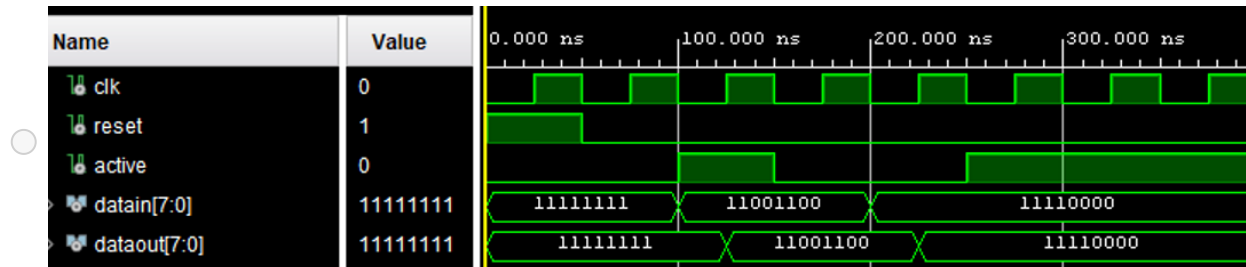
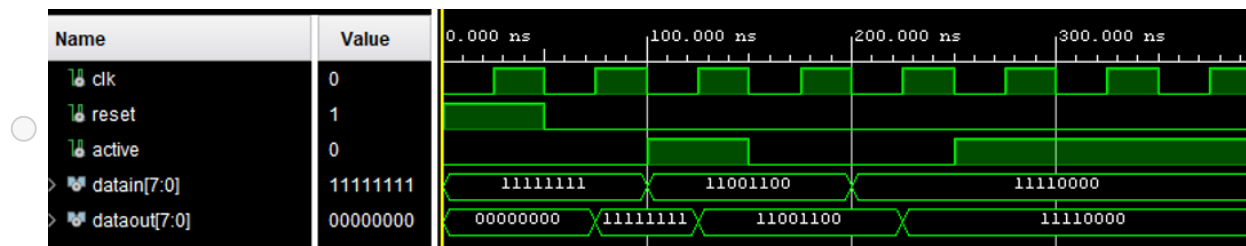
entity FF_Latch is
    Port ( clk, reset, active : in STD_LOGIC;
          datain : in SIGNED(7 downto 0);
          dataout : out SIGNED(7 downto 0) );
end FF_Latch;

architecture Behavioral of FF_Latch is
    signal internalsig : SIGNED(7 downto 0) := "00001111";
begin
    sync: process (clk, reset) is
    begin
        if reset = '1' then
            dataout <= "00000000";
        elsif (clk'event and clk = '1') then
            dataout <= internalsig;
        end if;
    end process;

    async: process (active, datain) is
    begin
        if active = '1' then
            internalsig <= datain;
        end if;
    end process;
end Behavioral;

```

Select one alternative:



Maximum marks: 2

3 Resulting value (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the VHDL code below. Assume that we assign the values '0', 'W', and 'Z' to insig1, insig2, and insig3, respectively. What is the resulting value on outsig?

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ResolTest is
    Port ( insig1, insig2, insig3 : in STD_ULOGIC;
          outsig : out STD_LOGIC );
end ResolTest;

architecture Behavioral of ResolTest is
begin
    outsig <= insig1;
    outsig <= insig2;
    outsig <= insig3;
end Behavioral;
```

Select one alternative:

☐ 'Z'

☒ '0'

☐ 'W'

Maximum marks: 2

4 Gate operation 1 (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the following VHDL code, which alternative gives the number of AND-gates in the implemented circuit?

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Gate_operation is
    Port ( a : in STD_LOGIC_VECTOR(15 downto 0);
          b : in STD_LOGIC_VECTOR(7 downto 0);
          res1 : out STD_LOGIC_VECTOR(7 downto 0);
          res2 : out STD_LOGIC_VECTOR(7 downto 0) );
end Gate_operation;

architecture Behavioral of Gate_operation is
begin
    COMP: for i in b'range generate
        res1(i) <= a(i) and b(i);
        res2(i) <= a(b'left + 1 + i) and b(i);
    end generate;
end Behavioral;
```

Select one alternative:

- ☐ 2
- ☐ 24
- ☒ 16

Maximum marks: 2

5 Gate operation 2 (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the VHDL code in the previous task (Gate operation 1), which alternative is the resulting value on *res2* if we assign "0000000010101010" to *a* and "11111111" to *b*?

Select one alternative:

☐ "11111111"

☐ "10101010"

☒ "00000000"

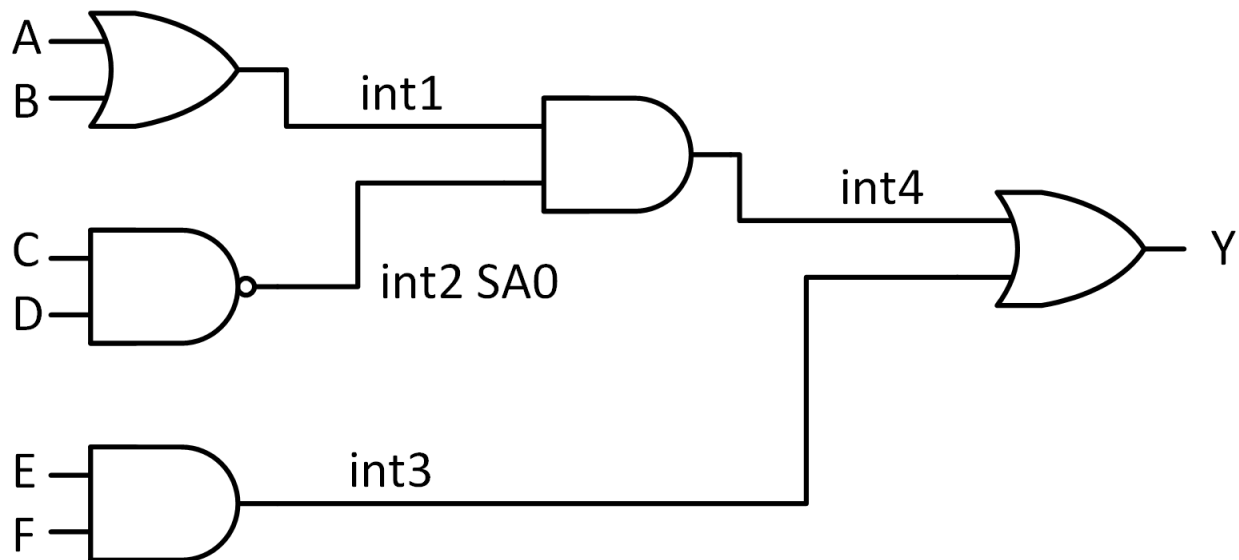
Maximum marks: 2

6 Test pattern (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Assume that we have found the following test pattern in order to detect signal int2 SA0 in the figure below:

ABCDEF = 101010

Which **other single** stuck-at-faults can we detect with the same pattern?



Select one alternative:

☐ int4 SA0, Y SA0

☐ The pattern only detects int2 SA0

☒ A SA0, D SA1, int1 SA0, int4 SA0, Y SA0

Maximum marks: 2

7 Project planning (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

When you during project planning are estimating how long a given task takes you can

Select one alternative:

- ☐ Only play planning poker
- ☒ Play planning poker and use a three point estimate
- ☐ Only use a three point estimate

Maximum marks: 2

8 Subtype (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Assume that you in your code have defined the following subtype:

subtype digits is integer range 0 to 9;

and declared the following signals:

signal MyDigit : digits;

What happens if Mydigit later is assigned the value 12.

Select one alternative:

- ☐ Nothing in particular happens
- ☐ You will not be able to synthesize the design
- ☒ The simulator will give you a warning or error message

Maximum marks: 2

9 State machines (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Given the following three statements about state machines.

A state machine is often used to control a data path.

A state machine should be described in VHDL using one single process.

The next state of state machine is found using a combinatorial circuit with the current state and current primary outputs as inputs.

How many of these statements are true?

Select one alternative:

☒ 1

☐ 3

☐ 2

Maximum marks: 2

10 Energy efficient design (Correct answer 2 points, wrong answer -1 point, no answer 0 points)

Which one of the following statements related to energy efficient design is **true**:

Select one alternative:

☒ Pipelining can make it possible to reduce the energy consumption of a circuit.

☐ Run-to-idle saves more energy than Dynamic voltage and frequency scaling.

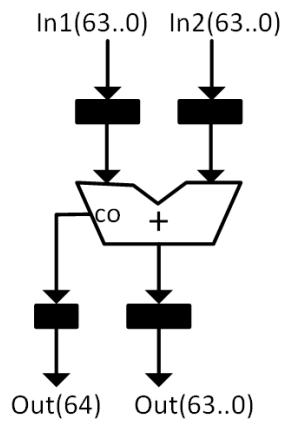
☐ Reducing the switching activity in a circuit reduces static energy consumption.

Maximum marks: 2

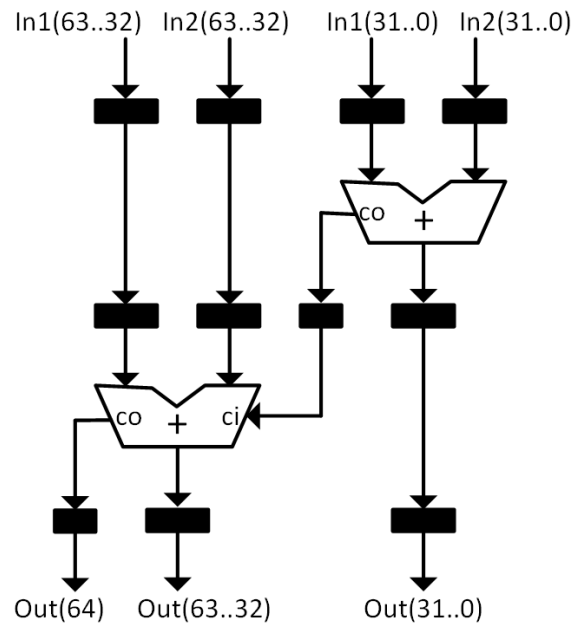
12 Adder (maximum 10 points)

Your customer wants you to design a low power / low energy adder for use in an ASIC. You are considering two alternative implementations as shown in the figure below. What are the advantages and disadvantages of the two alternatives? Explain the reasons for your answer considering the different sources for power and energy consumption.

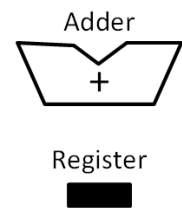
Area and performance (throughput and latency) are secondary objectives in the design. How do the two alternatives compare with respect to this?



IMPLEMENTATION 1



IMPLEMENTATION 2



Fill in your answer here

Format

 Σ |

Our low energy / power adder should run at a low clock frequency and low supply voltage. It should also have as low activity as possible, avoiding a lot of unnecessary glitches in the logic. Finally, it should have as few transistors as possible, since static energy and power consumption is directly related to this.

Implementation 1 has a very long critical path, especially if a carry propagate adder is used. This means that, for a given clock frequency, the supply voltage must be high enough to enable all input signals to propagate to the outputs. In Implementation 2 the critical path is shorter (about half for a carry propagate adder). With the same clock frequency, we can hence reduce the supply voltage substantially. We could also use a carry look ahead adder to reduce the critical path, but this would increase the number of gates and transistors, and hence the static consumption.

The extra registers needed for Implementation 2 will increase both static and dynamic energy and power, but the increase is most likely smaller than the decrease due to voltage scaling. The registers also enable pipelining, so that you can start a new addition before the previous is finished. If you, instead of reducing the supply voltage, increase the clock frequency to what the reduced critical path allows, you can achieve higher throughput. The latency in number of clock cycles will increase, but not necessarily in time, since each clock cycle is shorter.

Words: 0

Maximum marks: 10

Feel free to use your own RSA design as an example, but you can also base your answer on a fictitious design.

Fill in your answer here

Format ▾ | **B** *I* U x_2 x^2 | I_x |   |   |   |   |  |

Σ | 

A verification plan should cover all design steps:

- Write a high level model of the chosen algorithm and verify its functionality. Ideally this should result in a "golden model" of the design, with input patterns and expected resulting values, which can be used later in the design.
- The RSA circuit will typically be designed bottom up. The correct behavior of each sub module should then be verified with a separate test bench.
- The sub modules can be included in combined modules in an hierarchical way, verifying each new hierarchical level with extended test benches.
- The complete algorithm can be verified using the golden model of the high level model.
- Finally, the test bench included in the integration kit can be used to verify that the implementation with external interface behaves according to the specification, and the complete system functionality can be verified on the FPGA.

The entity on the next page is a combinatorial implementation of the modulo operation needed for the RSA implementation. The test bench is a very simple first version just to verify functionality with a limited set of values. It uses assertions to check correct results using the built in modulo operation, which is hard or impossible to synthesize. The test bench can easily be extended to read predefined input values from file, e.g. corner cases, or generate pseudo random input values.

Words: 0

Words: 0

Maximum marks: 18

```

entity ModuloOp is
    generic ( C_block_size : integer := 256 );
    port ( Pin : in UNSIGNED(C_block_size + 1 downto 0);
          Nin : in UNSIGNED(C_block_size - 1 downto 0);
          Pout : out UNSIGNED(C_block_size - 1 downto 0) );
end ModuloOp;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity TestbenchModulo is
end TestbenchModulo;

architecture Behavioral of TestbenchModulo is
    constant C_block_size : integer := 256;
    signal Pin : UNSIGNED(C_block_size + 1 downto 0);
    signal Nin : UNSIGNED(C_block_size - 1 downto 0);
    signal Pout : UNSIGNED(C_block_size - 1 downto 0);

begin
    DUT: entity work.ModuloOp(Behavioral)
        Generic map ( C_block_size )
        Port map ( Pin => Pin, Nin => Nin, Pout => Pout );

    stimulus: process is -- Simple first test with limited values
    begin
        Pin <= TO_UNSIGNED(8435, C_block_size + 2);
        Nin <= TO_UNSIGNED(9000, C_block_size);
        wait for 100 ns;
        assert (Pout = (Pin mod Nin)) -- Check correct value with mod operator (can not be synthesized)
            report "Error calculating modulo" severity warning;
        Pin <= TO_UNSIGNED(8435, C_block_size + 2);
        Nin <= TO_UNSIGNED(8000, C_block_size);
        wait for 100 ns;
        assert (Pout = (Pin mod Nin))
            report "Error calculating modulo" severity warning;
        Pin <= TO_UNSIGNED(8435, C_block_size + 2);
        Nin <= TO_UNSIGNED(3000, C_block_size);
        wait for 100 ns;
        assert (Pout = (Pin mod Nin))
            report "Error calculating modulo" severity warning;
        Pin <= TO_UNSIGNED(8435, C_block_size + 2); -- P > 3N-1
        Nin <= TO_UNSIGNED(1000, C_block_size); -- Check with illegal N-value compare to P
        wait for 100 ns;
        assert (Pout = (Pin mod Nin))
            report "Error calculating modulo" severity warning;
        wait;
    end process;
end Behavioral;

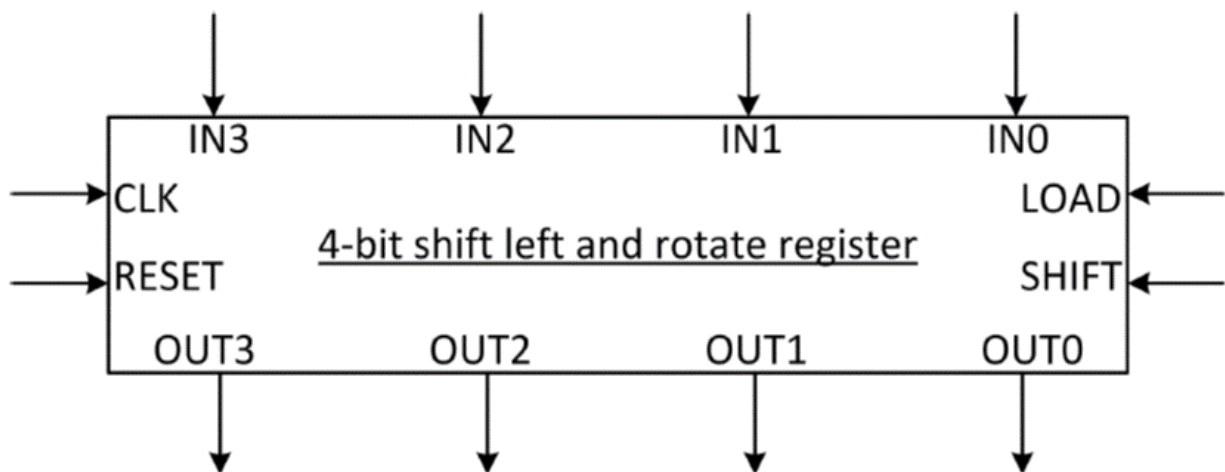
```

14 Design task (maximum 44 points)

You are going to design a digital system consisting of a application specific shift register and a state machine. The shift register has the following specification:

- Parallel load of data when the signal LOAD is active high ('1').
- Shift left of data (from least significant to most significant when the signal SHIFT is active high ('1')).
- Fixed rotation , so that data being shifted out from the most significant bit is shifted into the least significant bit.
- LOAD is to have priority over SHIFT. If both are active high, parallel data is to be loaded, without any shifting.
- Output data is to be presented in parallel continuously.
- The shift register shall have an active high ('1') asynchronous reset.

The following figure depicts the interface of the shift register.



a) (maximum 4 points)

Complete the following table (make a similar table in your answer box) according to the specification of the shift register:

SHIFT	LOAD	FUNCTION
0	0	Keep data unchanged
0	1	Load parallel data
1	0	Shift data left (and rotate)
1	1	Load parallel data

b) (maximum 8 points)

Draw a microarchitecture of the shift register, using, for instance, multiplexers and D-flipflops. Use the drawing option in the answer box or make a hand drawing on a separate sheet of paper to be marked with the control code and handed in at the end of the exam.

c) (maximum 8 points)

Write behavioral VHDL code that implements the shift register.

d) (maximum 8 points)

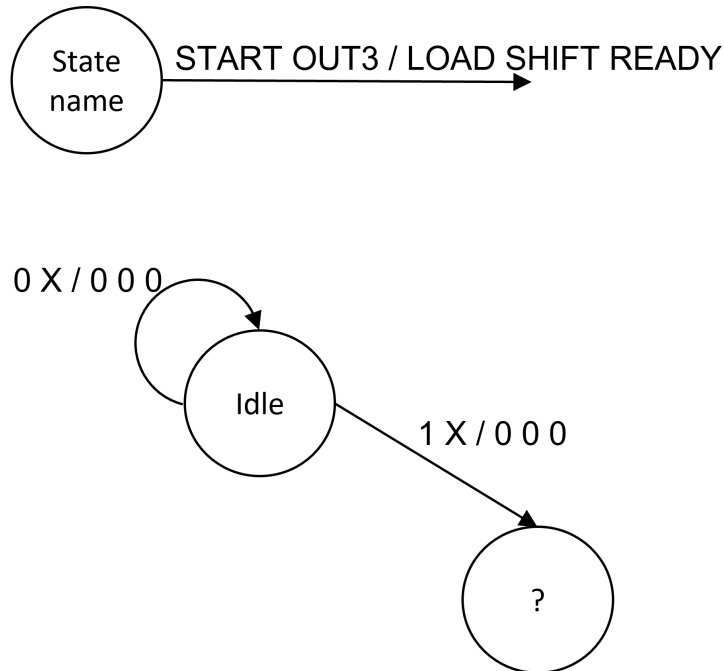
The overall system has the following specification:

- Stay in an idle state until an external signal START is active high ('1').
- Then load new parallel data, IN(3..0), into the shift register.

- Shift data left until the first '1' appears on OUT3. If IN3 is '1' no shifting shall take place.
- Set a READY signal high ('1').
- READY should be kept high for one clock period. Then return to the idle state if START is low ('0'), or load new data if START is high ('1').
- The value "0000" will never occur on IN(3..0).

Draw a state diagram for a state machine controlling the shift register in such a way that the overall system specification is fulfilled. Use the format indicated in the figure below. The idle state and a transition to another state is already drawn in the figure to show how it may look.

Use the drawing option in the answer box or make a hand drawing on a separate sheet of paper to be marked with the control code and handed in at the end of the exam.

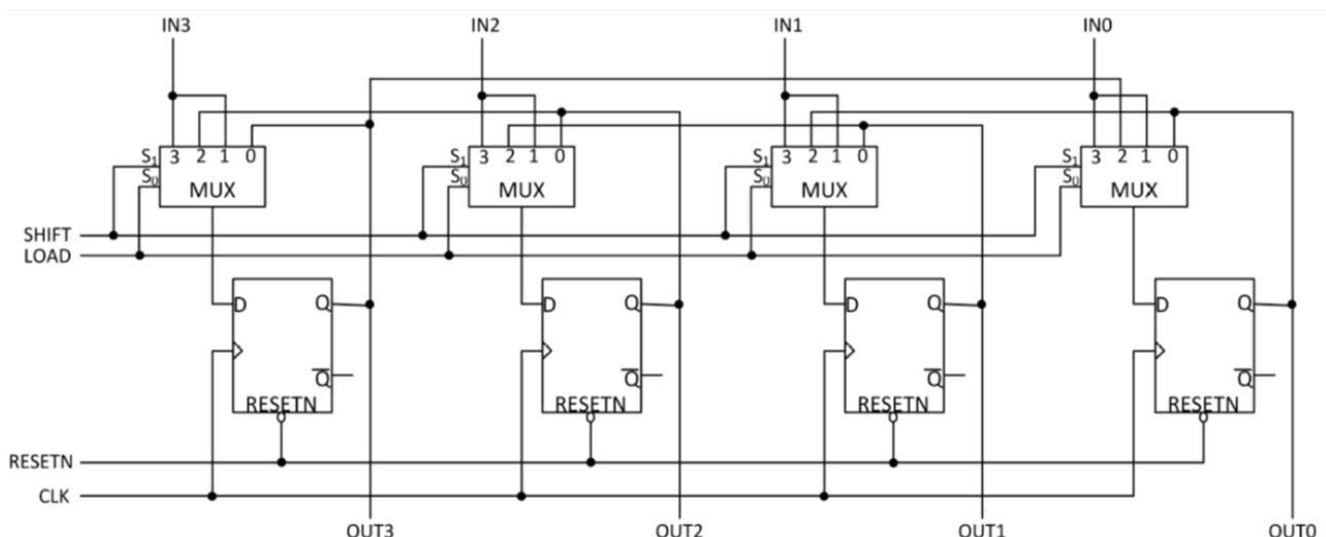


e) (maximum 8 points)

Write behavioral VHDL code that implements the state machine.

f) (maximum 8 points)

Write structural VHDL code, instantiating the entities of the shift register and state machine, to implement a system that fulfills the overall system specifications.



b): Micro architecture for shift register

c) Behavioral code for shift register

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ShiftReg is
    port ( clk, reset, shift, load : in std_ulogic;
          parin : in std_ulogic_vector(3 downto 0);
          parout : out std_ulogic_vector(3 downto 0) );
end ShiftReg;

architecture Behavioral of ShiftReg is
    signal internal_in, internal_out : std_ulogic_vector(3 downto 0);
begin

    mux_op: process (parin, internal_out, shift, load) is
    begin
        if load = '1' then
            internal_in <= parin;
        elsif shift = '1' then
            internal_in <= internal_out(2 downto 0) & internal_out(3);
        else
            internal_in <= internal_out;
        end if;
    end process;

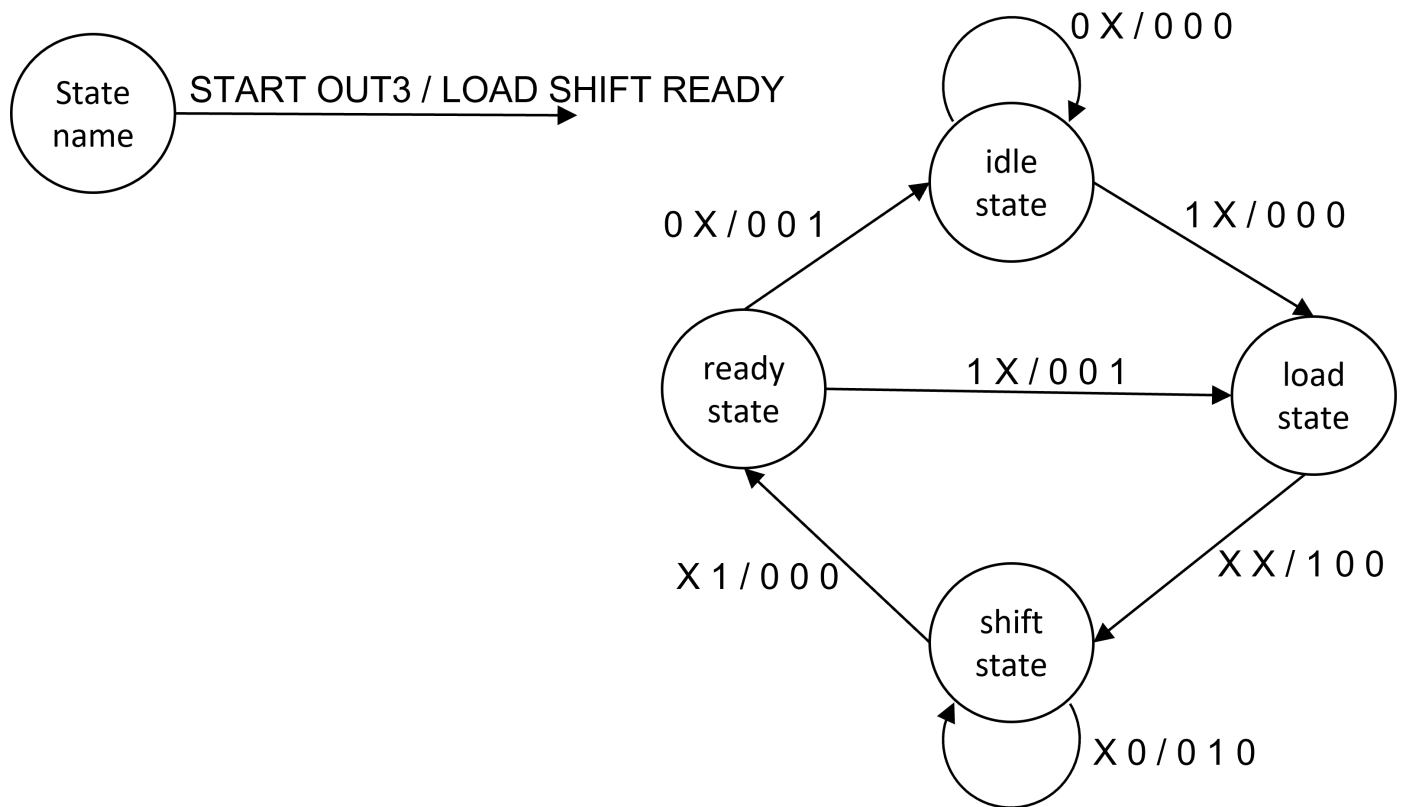
    dff_op: process (clk, reset) is
    begin
        if reset = '1' then
            internal_out <= "0000";
        elsif (clk'event and clk = '1') then
            internal_out <= internal_in;
        end if;
    end process;

    parout <= internal_out;

end Behavioral;
```

Note that we could also have implemented the "mux operation" as additional if-tests within the clocked process.

d) State diagram for state machine



Note that we have chosen to ignore any START signals coming while we are processing current data.

e) Implementation of state machine

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity StateMachine is
    port ( clk, reset, start, out3 : in std_ulogic;
          load, shift, ready : out std_ulogic );
end StateMachine;

architecture Behavioral of StateMachine is
    type state is (idlestate, loadstate, shiftstate, readystate);
    signal curr_state, next_state : state;
begin

    comb_proc: process (curr_state, start, out3) is
    begin
        load <= '0';
        shift <= '0';
        ready <= '0';
        case (curr_state) is
            when idlestate =>
                if start = '1' then
                    next_state <= loadstate;
                else
                    next_state <= idlestate;
                end if;
            when loadstate =>
                load <= '1';
                next_state <= shiftstate;
            when shiftstate =>
                if out3 = '1' then
                    next_state <= readystate;
                else
                    shift <= '1';
                    next_state <= shiftstate;
                end if;
            when readystate =>
                ready <= '1';
                if start = '1' then
                    next_state <= loadstate;
                else
                    next_state <= idlestate;
                end if;
            when others =>
                next_state <= idlestate;
            end case;
        end process;

    Synch_proc: process (clk, reset) is
    begin
        if reset = '1' then
            curr_state <= idlestate;
        elsif (clk'event and clk = '1') then
            curr_state <= next_state;
        end if;
    end process;

end Behavioral;
```

f) Implementation of structural version of full system

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FullSystem is
    port ( clk, reset, start : in std_ulogic;
          ready : out std_ulogic;
          parin : in std_ulogic_vector(3 downto 0);
          parout : out std_ulogic_vector(3 downto 0) );
end FullSystem;

architecture Structural of FullSystem is
    signal internal_out : std_ulogic_vector(3 downto 0);
    signal load, shift : std_ulogic;
begin

    STM: entity work.StateMachine(Behavioral)
        Port map ( clk => clk, reset => reset, start => start, out3 => internal_out(3),
                  shift => shift, load => load, ready => ready );

    SHFTREG : entity work.ShiftReg(Behavioral)
        Port map ( clk => clk, reset => reset, shift => shift, load => load,
                  parin => parin, parout => internal_out );

    parout <= internal_out;

end Structural;
```